

CH4 - Clip 10.

Airflow 실습 (Workflow Management)

01

실습 소개

02

실습 환경 구축

03

Airflow에서 Dag란?

04

실습

1. 실습 소개

Apache Airflow

기능 및 특징

- **정의 및 기능:** Apache Airflow는 Python으로 작성된 오픈 소스 워크플로우 관리 도구입니다. 데이터 엔지니어링 및 머신러닝 파이프라인의 스케줄링과 관리에 탁월
- **Directed Acyclic Graphs (DAGs):** 작업 간 의존성을 DAG 형태로 정의하여 복잡한 워크플로우를 효과적으로 관리
- **동적 파이프라인 생성:** Python 스크립트를 사용하여 동적으로 워크플로우를 생성하고 조정
- **장성과 유연성:** 다양한 Executor 지원으로 분산 환경에서의 확장성을 제공

MLOps에서의 활용 예시

- **데이터 처리 및 ETL 작업:** 대규모 데이터셋의 정제, 변환 및 로딩 작업을 자동화하고 스케줄링
- **모델 트레이닝 파이프라인:** 머신러닝 모델의 트레이닝 과정을 자동화하고, 데이터 전처리에서 모델 트레이닝, 결과 평가까지 일련의 단계를 관리



Airflow DAG 실습

Airflow DAG 생성 실습

- Hello dag 실습
- ML LifeCycle workflow를 management하는 dag 실습

2. 실습 환경 구축

실습 환경 구축

실습 환경 구축

- Pip install을 통해 local에 설정하기
- Docker container를 사용해서 docker image로 실행하기 (독립적인 공간에서 수행)

실습 환경 구축

실습 환경 구축

- Docker Image 생성을 진행
- Dockerfile을 정의
- `docker build -t my_airflow_image .`
- `docker run --name my_airflow_image -d -p 8080:8080 my_airflow_image:latest`

실습 환경 구축

실습 환경 구축

- Docker Image 생성을 진행
- Dockerfile을 정의
- `docker build -t my_airflow_image .`
- `docker run --name my_airflow_image -d -p 8080:8080 my_airflow_image:latest`

Dockerfile

```
# Dockerfile
FROM python:3.8-slim

# 환경 변수 설정
ENV AIRFLOW_HOME=/usr/local/airflow

# 필요한 시스템 라이브러리 설치
RUN apt-get update && \
    apt-get install -y gcc libc-dev vim && \
    rm -rf /var/lib/apt/lists/*

# Airflow 설치
RUN pip install apache-airflow

# Airflow 환경 설정
RUN mkdir -p $AIRFLOW_HOME
WORKDIR $AIRFLOW_HOME
RUN airflow db init

# Airflow DAGs 폴더에 DAG 파일 복사
COPY my_dag.py $AIRFLOW_HOME/dags/

# Airflow 웹 서버 포트 노출
EXPOSE 8080

# Airflow 웹 서버 및 스케줄러 실행
CMD airflow webserver -p 8080 & airflow scheduler
```

실습 환경 구축

실습 환경 구축 (계정 생성)

- Docker ps
- Docker exec -it [container id 또는 이름] bash
- Airflow users create \
 --username [사용자명] \
 --firstname [이름] \
 --role Admin \
 --email [이메일 주소]

<http://localhost:8080>

The screenshot displays the Apache Airflow web interface. At the top, there's a navigation bar with links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. Below this, two yellow warning banners are visible: 'Do not use SQLite as metadata DB in production' and 'Do not use the SequentialExecutor in production'. The main content area is titled 'DAGs' and includes a filter bar with buttons for 'All' (49), 'Active' (1), 'Paused' (48), 'Running' (1), and 'Failed' (0). There's also a search bar and an 'Auto-refresh' toggle. The DAGs are listed in a table with columns: DAG, Owner, Runs, Schedule, Last Run, Next Run, and Recent Tasks. The table shows various DAGs including dataset_consumes_1, dataset_produces_1, and example_branch_operator, each with its own status and schedule details.

3. Airflow에서 Dag란

Airflow의 DAG?

DAG 정의

- **DAG (Directed Acyclic Graph)**이며 Airflow에서 워크플로우를 정의하는 주요 구성 요소. DAG는 방향성을 가진 비순환 그래프이며, 여러 작업(Task)들과 이들 간의 의존성을 나타냄
- 작업 (Task) : DAG 내에서 실행되는 개별 단위. 예를 들어, 데이터를 다운로드하거나 변환/모델 학습 등의 작업이 있을 수 있음
- 의존성(Dependency) : 한 작업이 다른 작업에 의존하는 관계를 의미. 예를 들어, 데이터 변환 작업을 데이터 다운로드 작업이 완료된 후에 실행되어야 함

DAG 구성 요소

- **Operator** : Airflow에서 작업을 수행하는 객체. 다양한 유형의 Operator가 있으며, 각 각 특정 작업을 수행
- **Task** : Operator의 인스턴스로 DAG 내에서 실행되는 개별 작업을 나타냄
- **Task Instance** : 특정 시점에서 Task의 실행 인스턴스
- **Workflow** : 전체적인 작업의 흐름을 나타내며, 하나 이상의 DAG로 구성

Airflow의 DAG?

DAG 정의

- Dag는 Python 스크립트로 정의. 기본적인 구조는 이 코드와 같음

```
from airflow import DAG
from airflow.operators.python_operator import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'airflow',
    'start_date': datetime(2021, 1, 1),
    # 기타 필요한 기본 인자들 ...
}

dag = DAG(
    'example_dag',
    default_args=default_args,
    description='An example DAG',
    schedule_interval=timedelta(days=1),
)

def example_task():
    # Task에서 수행할 작업
    pass

task = PythonOperator(
    task_id='example_task',
    python_callable=example_task,
    dag=dag,
)
```

Airflow의 DAG?

작업 순서 및 의존성 설정

- 작업 순서와 의존성은 DAG 내에서 >> 또는 << 연산자를 사용하여 정의

```
task1 = PythonOperator(  
    task_id='task1',  
    python_callable=task1_function,  
    dag=dag,  
)  
  
task2 = PythonOperator(  
    task_id='task2',  
    python_callable=task2_function,  
    dag=dag,  
)  
  
# task1이 task2보다 먼저 실행되도록 설정  
task1 >> task2
```

Airflow의 DAG?

DAG 주의 사항

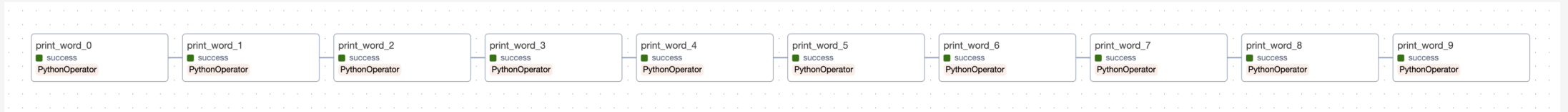
- **의존성 순환** : DAG에서는 순환 의존성을 가질 수 없음. 즉, 어떤 작업도 직접적이거나 간접적으로 자기 자신에게 의존할 수 없음
- **스케줄링** : 'start_date'와 'schedule_interval'을 적절히 설정하여 작업이 예상대로 실행되도록 해야함
- **오류 처리** : 각 Task는 실패할 수 있으므로, 오류 처리 로직을 고려해야함

4. Hello Airflow DAG 실습

Hello Airflow DAG Docker Container

실습 진행

- 1단계 : dag 작성
- 2단계 : dag 실행
- 3단계 : dag 실행 결과 확인

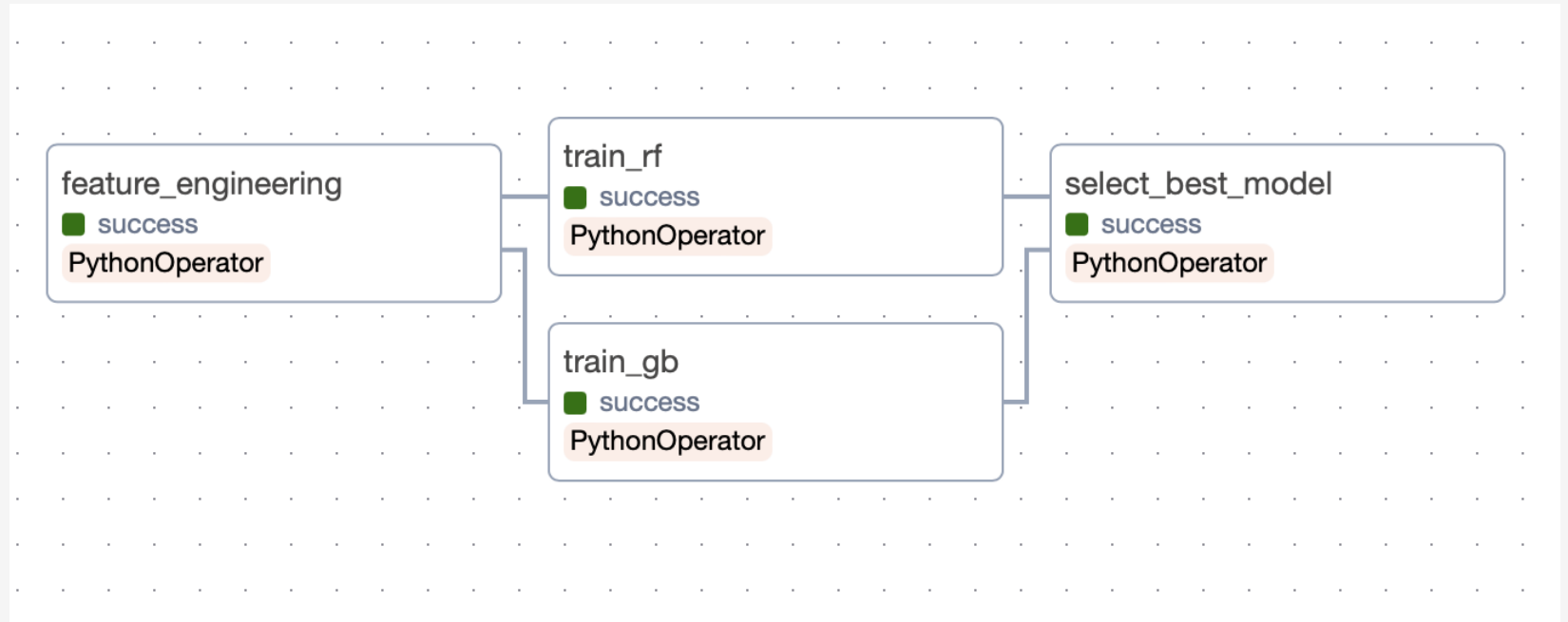


5. ML Development 과정의 DAG 실습

ML Development 과정 DAG 실습

실습 진행

- 1단계 : dag 작성
- 2단계 : dag 실행
- 3단계 : dag 실행 결과 확인



ML Development 과정 DAG 실습

실습 진행

- Pandas와 scikit-learn을 airflow container에 설치
- Airflow container bash로 연결 (docker exec --it [container id] bash)
- pip install pandas
- pip install scikit-learn